

3. Lab 3: Stacks & Queues

3.1. Objectives

- Know how to use the data structure Stack for solving real problems.

3.2. Problem 1: Simple stack application

Write a program to

- Convert a decimal number and convert it to octal form.
- Concatenate two stacks.
- Determine if the contents of one stack are identical to that of another.

3.3. Problem 3: Undo and redo

Write a class *SpecialArray* that has:

- (8 points) an array of 20 random values
- (8 points) a function to update the value at a position in the array.
- (8 points) a function to undo the updating.
- (8 points) a function to redo the updating.
- (8 points) a function to display content of the array.

Hint: use two stacks to store the array after each operation

3.4. QueueApp.java

- Write a method to display the queue array and the front and rear indices. Explain how wraparound works.
- Write a method to display the queue (loop from 1 to nItems and use a temporary front for wraparound).
- Display the array, the queue, and the front and rear indices.
- Insert fewer items or remove fewer items and investigate what happens when the queue is empty or full.
- Extend the insert and remove methods to deal with a full and empty queue.
- Add processing time to the queue. Create a new remove method that removes item N after N calls to the method.
- Simulate a queue of customers each one served for a random amount of time. Investigate how simulation is affected by:
 - the size of the queue
 - the range of time for which each customer is served
 - the rate at which customers arrive at the queue

3.5. ReverseApp.java

- Create a stack of objects of class Person and use to reverse a list of persons.

3.6. PriorityQApp.java

- Write a method to display the queue and use to trace the queue operation.
- Modify the insert method to insert the new item at the rear. Compare this queue with QueueApp.java. Which one is more efficient?
- Use a priority queue instead of an ordinary one in the simulation experiments described above.

3.7. Challenge

Suppose you have a deque D containing the numbers (1,2,3,4,5,6,7,8), in this order. Suppose further that you have an initially empty queue Q. Implement a function that uses only D and Q (and no other variables) and results in D storing the elements in the order (1,2,3,5,4,6,7,8).

Basic Operations:

- Deque D:
 - pushFront(x), pushBack(x), popFront(), popBack().
- Queue Q:
 - enqueue(x), dequeue().

Extended Problem

Reversing the Deque

1. **Original Setup:** D contains (1,2,3,4,5,6,7,8). Q is empty.
2. **Goal:** Reverse the order of D so that it becomes (8,7,6,5,4,3,2,1)
3. **Constraints:**
 - Only D and Q may be used.
 - No additional variables allowed.

Note: explain your algorithm in your report and show your results.